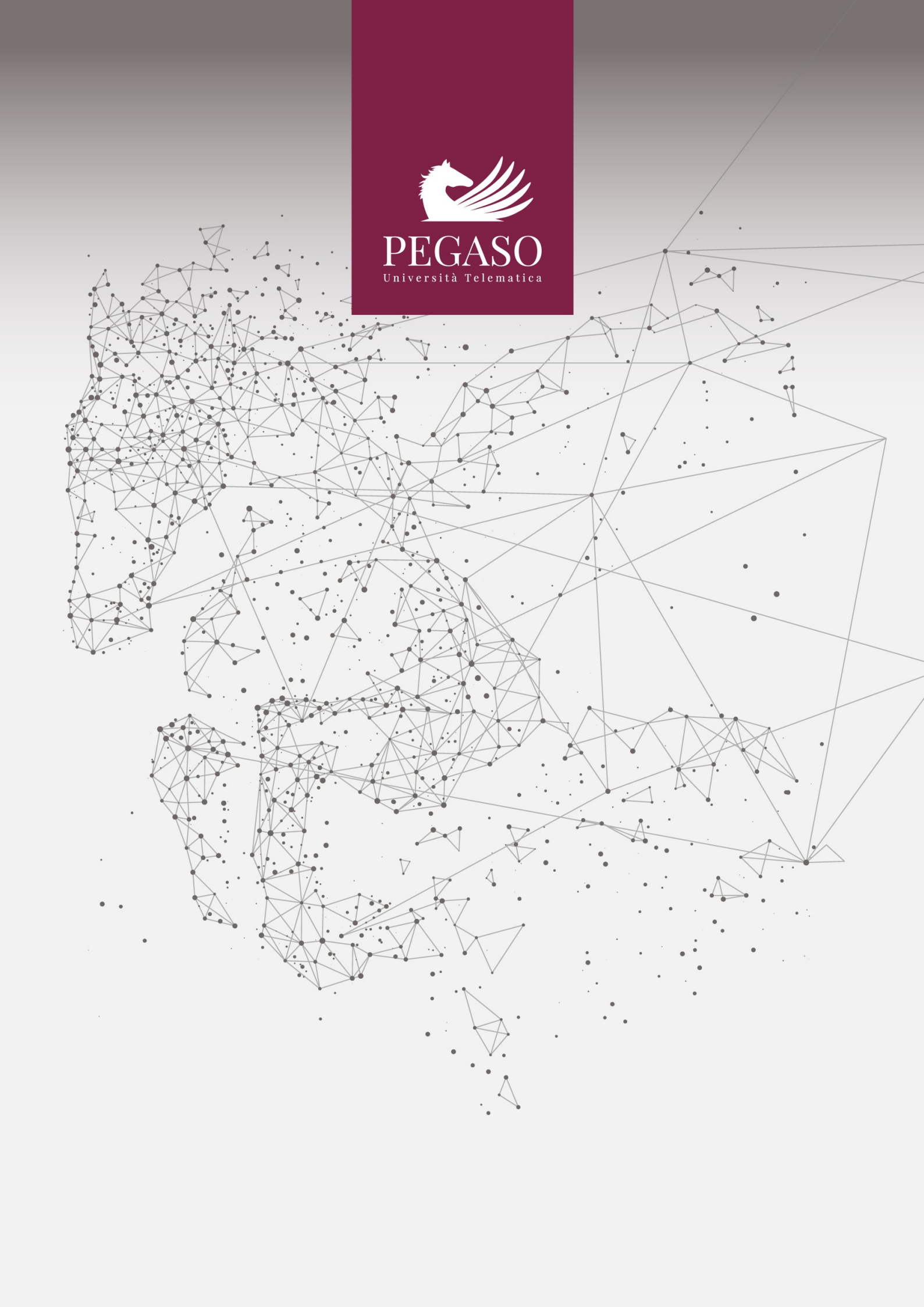

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. SYNC/ASYNC 3
2. WEBSOCKET 4
3. PROTOCOLLO WEBSOCKET 5
4. SOCKET.IO 7
5. IMPLEMENTAZIONE WEBSOCKET E SOCKET.IO 8
BIBLIOGRAFIA 13
SITOGRAFIA 14

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Sync/Async

In una programmazione sincrona, ogni operazione viene eseguita una dopo l'altra in un unico flusso di esecuzione: questo significa che, quando una funzione viene chiamata, il programma si ferma finché la funzione non ha completato l'operazione e restituito un risultato. Solo a questo punto il programma può passare alla funzione successiva.

In una programmazione asincrona, invece, le operazioni vengono eseguite in modo indipendente dal flusso principale di esecuzione del programma: quando una funzione viene chiamata, il programma non si ferma ma continua a eseguire altre operazioni, mentre la funzione viene eseguita in parallelo. Quando la funzione ha terminato l'operazione, il risultato viene restituito attraverso una callback o una Promise.

In JavaScript, la programmazione asincrona è fondamentale per gestire le operazioni che richiedono del tempo, come le richieste HTTP o le operazioni di lettura/scrittura su file system: grazie alla programmazione asincrona, il programma non si blocca durante queste operazioni ma può continuare a eseguire altre attività, migliorando la reattività dell'applicazione e l'esperienza utente.

Introduciamo ora i costrutti async / await:

- **async** è utilizzato per dichiarare una funzione asincrona, ovvero una funzione che restituisce una Promise; all'interno di una funzione dichiarata come async, si può utilizzare await per attendere il completamento di un'altra funzione asincrona o di una Promise: in questo modo si può scrivere codice asincrono in modo simile a quello sincrono;
- l'utilizzo di **await** sospende l'esecuzione della funzione asincrona in attesa del completamento della Promise: se la Promise viene risolta, await restituisce il valore restituito dalla Promise stessa; se invece la Promise viene rifiutata, viene sollevata un'eccezione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. WebSocket

I WebSockets sono una tecnologia per la comunicazione bidirezionale in tempo reale tra client e server, che consente di inviare e ricevere dati in modo efficiente e scalabile. A differenza delle tradizionali richieste HTTP, che utilizzano il modello client-server a singola richiesta-risposta, i WebSockets mantengono una connessione bidirezionale costante tra il client e il server.

I WebSockets sono introdotti attraverso il protocollo WebSocket, che viene stabilito tramite una richiesta HTTP di handshake tra il client e il server. Una volta stabilito il protocollo WebSocket, il client e il server possono inviare e ricevere dati in tempo reale attraverso una connessione bidirezionale costante.

Per utilizzare i WebSockets, è necessario scrivere codice sia lato client che lato server. Sul lato client, è possibile utilizzare la classe WebSocket, inclusa nel browser, per creare una connessione WebSocket. Sul lato server, è necessario utilizzare una libreria WebSocket specifica, come 'ws' per Node.js, per gestire la connessione WebSocket.

Una volta stabilita la connessione WebSocket, sia il client che il server possono inviare messaggi in qualsiasi momento. Per inviare un messaggio, è possibile utilizzare il metodo 'send' della classe WebSocket lato client e il metodo 'send' dell'oggetto di connessione WebSocket lato server.

Il protocollo WebSocket è uno standard web che può essere implementato in qualsiasi linguaggio di programmazione che supporta le API per le socket. Tuttavia, la maggior parte delle librerie per la gestione dei WebSockets sono scritte in JavaScript, poiché è il linguaggio di scripting utilizzato sui browser web.

Sul lato client, i WebSockets sono implementati in JavaScript attraverso la classe WebSocket del browser, che consente di creare e gestire una connessione WebSocket. Sul lato server, esistono diverse librerie WebSocket scritte in JavaScript, come ad esempio 'ws' per Node.js, 'Socket.IO' per Node.js e browser, 'SockJS' per Node.js e browser e molte altre.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Protocollo websocket

Analizziamo il protocollo nel dettaglio:

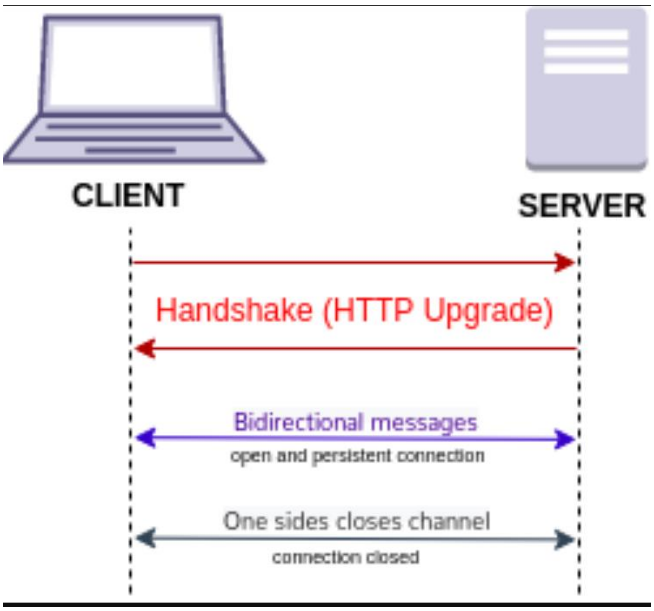


***Figura 1** - WebSocket Protocol*

- Il protocollo WebSocket utilizza il protocollo di handshake HTTP per stabilire la connessione. In particolare, il client invia una richiesta di handshake WebSocket al server come una richiesta HTTP GET con un campo "Upgrade" nell'header della richiesta impostato su "websocket". Il campo "Connection" dell'header viene impostato su "Upgrade", e il campo "Sec-WebSocket-Key" contiene un valore casuale generato dal client.
- Il server risponde con una risposta di handshake WebSocket, anch'essa come una risposta HTTP. Il campo "Upgrade" nell'header della risposta viene impostato su "websocket", il campo "Connection" viene impostato su "Upgrade", e il campo "Sec-WebSocket-Accept" contiene un valore derivato dal campo "Sec-WebSocket-Key" della richiesta di handshake del client. Il valore del campo "Sec-WebSocket-Accept" è generato utilizzando una funzione di hash SHA-1, una costante predefinita e il valore del campo "Sec-WebSocket-Key" della richiesta di handshake del client.
- Una volta completato il protocollo di handshake, la connessione WebSocket viene stabilita e il client e il server possono iniziare a inviare e ricevere messaggi. I messaggi inviati tramite il protocollo WebSocket sono divisi in frame, con un frame di intestazione e il payload dei dati del

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

messaggio. Il frame di intestazione contiene informazioni sul messaggio, come la lunghezza del payload e il tipo di dati contenuti nel payload.

- Il protocollo WebSocket prevede la possibilità di inviare dati di tipo binario o di tipo testo. Inoltre, il protocollo WebSocket prevede anche la possibilità di inviare messaggi suddivisi in più frame, utilizzando il frame di intestazione per specificare la lunghezza totale del messaggio.
- Il protocollo WebSocket prevede anche la possibilità di gestire gli errori durante la connessione. Ad esempio, se il server non risponde o se la connessione viene persa, il client può inviare un messaggio di chiusura per terminare la connessione in modo pulito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Socket.IO

Socket.IO è una libreria JavaScript per applicazioni web in tempo reale che fornisce un'astrazione sopra i WebSockets, consentendo una comunicazione bidirezionale in tempo reale tra client e server. Socket.IO utilizza una serie di tecniche, tra cui WebSockets, polling e long-polling, per fornire una connessione stabile e affidabile tra client e server.

Socket.IO è progettato per essere facile da usare e supporta numerosi ambienti di sviluppo, tra cui Node.js, browser web e dispositivi mobili. La libreria è disponibile come pacchetto NPM e può essere utilizzata in combinazione con altre librerie e framework JavaScript per creare applicazioni web in tempo reale complesse e scalabili.

Per utilizzare Socket.IO, è necessario configurare un server Socket.IO che ascolti le connessioni dei client e gestisca la comunicazione tra il client e il server. Inoltre, è necessario includere la libreria Socket.IO nel client HTML per consentire al browser di comunicare con il server Socket.IO. Una volta configurato il server e il client, è possibile utilizzare gli eventi e le funzioni di Socket.IO per inviare e ricevere dati in tempo reale tra client e server.

Socket.IO è stato sviluppato per semplificare la creazione di applicazioni web in tempo reale e risolvere le sfide legate alla comunicazione bidirezionale tra client e server. La libreria offre un'ampia gamma di funzionalità e supporta numerosi ambienti di sviluppo, rendendola una scelta popolare per gli sviluppatori che creano applicazioni web in tempo reale complesse e scalabili.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Implementazione websocket e Socket.IO

È possibile realizzare una implementazione di WebSocket direttamente in Replit.com usando Nodejs:

```
const express = require('express');
const WebSocket = require('ws');

const app = express();

const server = app.listen(3000, () => {
  console.log('Server web avviato');
});

const wsServer = new WebSocket.Server({ server });

wsServer.on('connection', (socket) => {
  console.log('Client connesso');

  socket.on('message', (message) => {
    console.log('Messaggio ricevuto:', message);

    // Invia il messaggio di risposta al client
    socket.send(`Hai detto: ${message}`);
  });

  socket.on('close', () => {
    console.log('Client disconnesso');
  });
});


// Crea il client WebSocket
const socket = new WebSocket('ws://localhost:3000');

socket.onopen = () => {
  console.log('Connesso al server WebSocket');
  socket.send('Ciao server WebSocket!');
};

socket.onmessage = (event) => {
  console.log('Messaggio ricevuto dal server:', event.data);
};
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
socket.onclose = () => {
  console.log('Connessione al server WebSocket chiusa');
};
```

Direttamente dalla shell di replit è possibile avviare il server usando il comando:

`node index.js`

Lo scambio sarà:

```
Server web avviato
Client connesso
Connesso al server WebSocket
Messaggio ricevuto: <Buffer 43 69 61 6f 20 73 65 72 76 65 72 20 57 65 62
53 6f 63 6b 65 74 21>
Messaggio ricevuto dal server: Hai detto: Ciao server WebSocket!
```

Questo codice mostra come creare un server WebSocket utilizzando la libreria ws e come connettersi ad esso utilizzando un client WebSocket.

Prima di tutto, vengono importate le librerie necessarie, ovvero Express e WebSocket. Viene quindi creato un'applicazione Express e un server viene avviato sulla porta 3000.

Viene quindi creato un nuovo server WebSocket e viene ascoltato l'evento 'connection'. Quando un nuovo client WebSocket si connette al server, viene emesso l'evento 'connection' e viene passato un oggetto socket che rappresenta la connessione.

Quando un messaggio viene inviato dal client, viene emesso l'evento 'message' e il messaggio viene registrato nella console del server. Viene quindi inviato un messaggio di risposta al client utilizzando il metodo socket.send.

Infine, viene creato un client WebSocket che si connette al server WebSocket utilizzando la stessa URL del server. Quando la connessione viene stabilita, viene inviato un messaggio di saluto al server. Quando il server invia un messaggio, viene registrato nella console del client.

Qualora si volesse costruire un'applicazione più articolata in Replit.com si avrebbero alcuni problemi in quanto Replit usa HTTPS come protocollo predefinito per le pagine web, mentre il server WebSocket creato ascolta sulla porta http; questo problema può essere risolto utilizzando il protocollo HTTPS per il server WebSocket generando un certificato SSL e configurando il server WebSocket per utilizzarlo, o in alternativa usando un servizio di terze parti che offra un certificato SSL e consenta di utilizzarlo per il server WebSocket. Questa operazione non è tuttavia di facile costruzione, per questo motivo è preferibile usare Socket.io.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Index.js

```
const express = require('express');
const app = express();
const httpServer = require('http').createServer(app);
const io = require('socket.io')(httpServer);

const PORT = process.env.PORT || 3000;

app.use(express.static(__dirname + '/public'));

io.on('connection', (socket) => {
  console.log('Nuova connessione');

  // Ricezione del messaggio dal client
  socket.on('message', (message) => {
    console.log('Messaggio ricevuto:', message);

    // Invio del messaggio di risposta all client
    socket.emit('message', `Hai detto: ${message}`);
  });

  socket.on('disconnect', () => {
    console.log('Client disconnesso');
  });
});

httpServer.listen(PORT, () => {
  console.log(`Server in ascolto sulla porta ${PORT}`);
});
```

Index.html

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebSocket Echo Client</title>
  </head>
  <body>
    <form id="form">
      <label for="message">Messaggio:</label>
      <input type="text" id="message" name="message">
      <button type="submit">Invia</button>
    </form>

    <div id="messages"></div>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    <script src="/socket.io/socket.io.js"></script>
    <script>
      const socket = io('wss://socketiotest.prof-stefanostefano.repl.co');

      // Gestione dell'invio del messaggio dal form
      const form = document.getElementById('form');
      form.addEventListener('submit', (event) => {
        event.preventDefault();
        const messageInput = document.getElementById('message');
        const message = messageInput.value;
        socket.emit('message', message);
        messageInput.value = '';
      });

      // Gestione del messaggio di risposta dal server
      socket.on('message', (message) => {
        const messagesContainer = document.getElementById('messages');
        const messageElement = document.createElement('p');
        messageElement.textContent = message;
        messagesContainer.appendChild(messageElement);
      });
    </script>
  </body>
</html>
```



***Figura 2** - Socket.io output*

Questo codice crea un'applicazione Node.js che utilizza la libreria Socket.IO per creare una connessione WebSocket tra un client e un server.

In particolare, l'applicazione server crea un'istanza di express e un server HTTP utilizzando il modulo http di Node.js. Quindi, viene creata un'istanza di Socket.IO passando il server HTTP creato in precedenza. L'applicazione utilizza la funzione express.static per servire il file index.html nella directory public.

Quando un client si connette al server WebSocket, il server registra l'evento "connection" e ascolta i messaggi inviati dal client. Quando il server riceve un messaggio dal client, invia un messaggio di risposta al client.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel codice HTML, viene creato un form che consente all'utente di inviare messaggi al server tramite la connessione WebSocket. Quando l'utente invia un messaggio, il client lo invia al server tramite la connessione WebSocket e attende una risposta dal server. Quando arriva la risposta, il client la visualizza nella pagina HTML.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).